

Class 13: Testing

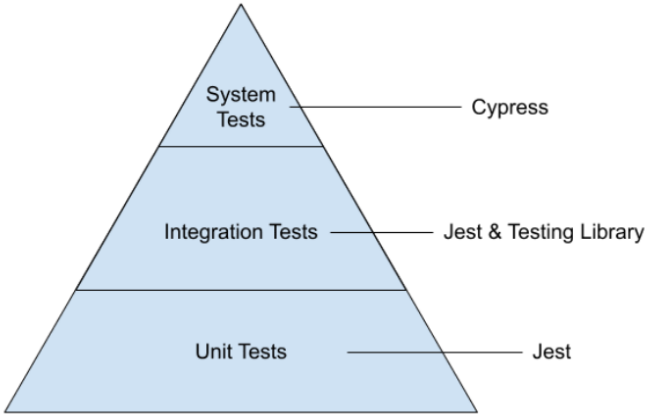
2110507 Software Development Practice II

Machigar Ongtang, Ph.D.

Testing

- **Unit Testing สำหรับแต่ละ component**
  - Test แต่ละ single unit เช่น functional unit ต่าง ๆ รวมถึง Class, Component ต่าง ๆ
  - แยกออกมา Test โดยไม่คำนึงถึง dependencies จากภายนอก หรือหากจำเป็นต้องมี สามารถสร้าง mock dependencies เพื่อจำลองการทำงานกับภายนอก

- **Integration Testing เป็นการ test ในระดับที่สูงขึ้นมา**
  - รวมเอาผลลัพธ์ของส่วนต่าง ๆ และ source ต่าง ๆ
  - ตรวจสอบการทำงานร่วมกันระหว่าง unit ต่าง ๆ ที่ทำงานประกอบกัน
  - ครอบคลุมหลายระดับ ไปจนถึง database หรือ server connection



- **End-to-End (E2E) Testing**
  - Evaluate การทำงานของทั้ง application flow ตั้งแต่ต้นจนจบ
  - Test ตาม Scenario การใช้งานจริง เพื่อให้มั่นใจว่า user สามารถใช้งานได้จริง และสามารถ interact กับ program ได้ตามที่คาดหวัง

Testing



- **Acceptance Testing** เป็นการ test การยอมรับการใช้งาน
  - เพื่อให้มั่นใจว่าสิ่งที่สร้างขึ้น ตอบ Business Requirements จริง ๆ
  - ทดสอบร่วมกันกับ User, Product Manager เพื่อให้ระบบเป็นที่ยอมรับโดย stakeholders ต่าง ๆ

- **Performance testing** เป็นการทดสอบประสิทธิภาพของระบบ
  - Performance testing ทดสอบ non-functional requirements ต่าง ๆ รวมถึง scalability, stability และ reliability ของระบบ

Testing Libraries & Tools



- Testing Libraries สำหรับ React โดยเฉพาะ Next.js มีค่อนข้างจำกัด
  - อาจต้องทดสอบ Compatibility ระหว่าง Testing Library Version กับ Next.js Version ที่ใช้งาน

- Jest และ React Testing Library เป็น JavaScript testing framework ที่มักใช้งานร่วมกัน โดยมาพร้อมกับ React utility สำหรับใช้ในการ test component

- Jest: เป็น Tools ที่โดยทั่วไปใช้ในการทำ unit test (https://jestjs.io/)
  - ให้ JavaScript testing framework มี built-in mocking tools
  - Jest เป็น open-source test runner ที่ค้นหา test ทำการ run test ต่าง ๆ รวมถึงพิจารณาว่า test นั้นผ่าน (pass) หรือ ไม่ผ่าน (failed)
  - Jest มาพร้อม function ต่าง ๆ สำหรับ test suite, test case และการทำ assertion
  - ทำงานกับ project ต่าง ๆ ทั้ง Babel, TypeScript, Node, React, Angular, Vue ฯลฯ

Testing Libraries & Tools



- **React Testing Library (RTL)** เป็น lightweight testing library สำหรับ React applications
  - ออกแบบให้ทำงานเฉพาะเจาะจงกับ Component ใน React Ecosystem
  - สร้าง virtual DOMs สำหรับ test React component
    - เนื่องจากการ test เกิดขึ้นโดยอัตโนมัติ ไม่ได้กระทำบน Web Browser
  - เน้นการ test พฤติกรรมของ application โดยเลียนแบบหรือ simulate การ interact ของ user กับ component มากกว่าการพิจารณารายละเอียดของการ implementation
    - รองรับการสร้าง events ต่าง ๆ เช่น clicking, double-clicking, typing, hovering ฯลฯ
  - มาพร้อม function ที่ใช้ในการ render component และทดสอบการเปลี่ยนแปลงของ UI ที่เกิดขึ้นบน screen เมื่อมีการทำ interaction
    - ทำการ re-render component เมื่อมี user interaction
    - ทำการ clean และ unmount React tree หลังการทำ execution เพื่อป้องกัน memory leaks
    - มาพร้อม method ต่าง ๆ เช่น query method เพื่อเข้าถึง component
  - https://testing-library.com/

Jest



- โครงสร้างของ test method
  - ใช้เมธอด `test` หรือ alias ว่า `it`

```
test(name, function, [timeout])
```

```
it(name, function, [timeout])
```

  - *name* ชื่อของการ test
  - *function* Function บรรจุการทำงานของสิ่งที่จะ test และความคาดหวัง (expectation) ในผลลัพธ์ของการ test
  - *[timeout]* ระยะเวลาที่ต้องการให้รอ ก่อนที่จะ abort การทดสอบ (ms) ค่า default เป็น 5 s

```
it('should appear', () => {
  // … test code here …
});
```

  - หากมีการ return promise จาก test, Jest จะรอให้ promise ถูก resolve ก่อนจบ test แม้ test จะ return ทันที แต่ test จะยังไม่จบจนว่า promise จะ resolve

return promise

```
it('has lemon in it', () => {
  return fetchList().then(list => {
    expect(list).toContain('lemon');
  });
});
```

Jest



- โครงสร้างของ function ใน test

  โดยทั่วไป มักมี 3 องค์ประกอบ

  - Arrange: Code สำหรับ setup สิ่งที่ต้องการทดสอบ
  - Act: Code สำหรับกระทำ action บนสิ่งที่ setup เอาไว้ หรือ Query ดูรายละเอียดเพื่อสังเกตุที่ที่เกิดขึ้น
  - Assert: Code สำหรับตรวจสอบว่าสิ่งที่คาดหวัง เป็นจริงหรือไม่

```
it('should appear', () => {
  // ARRANGE
  // … code to arrange environment …
  // ACT
  // … code to perform action in environment
  // or query what are in environment …
  // ASSERT
  // … code to check the fact of the result
});
```

- การสร้างกลุ่มของการ test
  - ใช้เมธอด `describe` ในการสร้าง group ของ test ที่เกี่ยวข้องกัน
  - *`blockName`* ชื่อ block หรือกลุ่ม test

```
describe(blockName, ()=>{
   it(…, () => { … })

   it(…, () => { … })
})
```

Jest



- การทำ Assertion เพื่อตรวจสอบผลการ test
  - ใช้ในการทดสอบค่าว่า value ของผลลัพธ์เข้ากับ condition ที่คาดหวังไว้หรือไม่
  - ใช้เมธอด `expect` ในการทดสอบผ่าน matcher ประเภทต่าง ๆ สำหรับ validate สิ่งที่เกิดขึ้น
  - https://jestjs.io/docs/expect

```
expect(value)
```

  - *value* ค่าที่ต้องการ assert ว่าตรงเงื่อนไขหรือไม่
- โดยปกติจะใช้ `expect` ควบคู่กับ matcher เพื่อใช้ในการระบุเงื่อนไขการทดสอบ
  - Matcher `.toBe(value)` ใช้ในการเปรียบเทียบความเท่ากัน ผ่าน `Object.is`

```
const can = {
  name: 'pamplemousse',
  ounces: 12,
};

describe('the can', () => {
  it('has 12 ounces', () => {
    expect(can.ounces).toBe(12);
  });
});
```

Jest



- Assertion Matcher
  - Matcher `.toHaveBeenCalled()` หรือ `.toBeCalled()` ในการตรวจสอบว่า function ถูกเรียกหรือไม่

```
function drinkAll(callback, flavour) {
  if (flavour !== 'octopus') {
    callback(flavour);
  }
}
describe('drinkAll', () => {
  test('drinks something lemon-flavoured', () =>
  {
    const drink = jest.fn();
    drinkAll(drink, 'lemon');
    expect(drink).toHaveBeenCalled();
  });
});
```

  - Matcher `.toBeTruthy()` ตรวจสอบว่าค่าเป็น `true` หรือไม่
  - Matcher `.toBeFalsy()` ตรวจสอบว่าค่าเป็น `false` หรือไม่
  - Matcher `.toBeNull()` ตรวจสอบว่าค่าเป็น `null` หรือไม่

```
it('Status should be true', () => {
  const status = …
  expect(status).toBeTruthy();
});
```

Jest



- Matcher `.toContain(item)` ตรวจสอบว่า array บรรจุค่า *item* หรือไม่

```
it('the flavor list contains lime', () => {
  expect(getAllFlavors()).toContain('lime');
});
```

- Matcher `.toHaveReturned()` หรือ `.toReturned()` ในการตรวจสอบว่า function มีการ return หรือไม่

```
it('drinks returns', () => {
  const drink = jest.fn(() => true);
  drink();
  expect(drink).toHaveReturned();
});
```

- Matcher `.toHaveReturnedWith(value)` หรือ `.toReturnedWith(value)` ในการตรวจสอบว่า function มีการ return ค่า *value* ที่ระบุ หรือไม่

```
it('drink returns La Croix', () => {
  const beverage = {name: 'La Croix'};
  const drink = jest.fn(beverage =>
beverage.name);

  drink(beverage);
  expect(drink).toHaveReturnedWith('La Croix');
});
```

Jest



- Assertion Modifier ใช้ในการปรับรูปแบบการทำ assertion
  - Modifer `.not` ใช้ในการ assert สิ่งที่ตรงกันข้าม

```
it('best flavor is not coconut', () => {
  expect(bestFlavor()).not.toBe('coconut');
});
```

  - Modifer `.resolves` ใช้ในการ unwrap ค่าของ promise ที่ได้รับการ fulfill แล้ว เพื่อให้ chain เข้ากับ matcher ตัวอื่นได้ หาก promise ถูก reject จะทำให้ assertion นั้น fail

```
test('resolves to lemon', () => {
   return expect(Promise.resolve('lemon')).resolves.toBe('lemon');
});
```

  - Modifer `.rejects` ใช้ในการ unwrap ค่าของ promise ที่ได้รับการ reject เพื่อให้ chain เข้ากับ matcher ตัวอื่นได้ หาก promise ถูก fulfill จะทำให้ assertion นั้น fail

```
test('rejects to octopus', () => {
  return expect(Promise.reject(new Error('octopus'))).rejects.toThrow('octopus');
});
```

Jest



- **Jest Mock Functions** ใช้ในการสังเกตพฤติกรรมของ function แต่ถูกเรียกจาก code อื่น ไม่ได้สนใจเพียงผลลัพธ์ output
  - สามารถสร้าง mock function ด้วย `jest.fn()`
  - ถ้าไม่ใส่ implementation เข้าไปใน mock function จะทำให้ mock function นั้นคืนค่าเป็น undefined

```
const mockFn = jest.fn( implementation );
```

- *implementation* Function Implementation
- *mockFn* ชื่อ mock function ที่สร้างได้

```
const mockFn = jest.fn((scalar: number) =>
                    42 + scalar);
```

```
mockFn(0); // 42
mockFn(1); // 43
```

```
const mockFn = jest.fn.mockImplementation((scalar: number) =>
                    42 + scalar);
```

Jest



- Mock ค่า Return Value

```
const mockFn = jest.fn().mockReturnValue(value);
```

  - มีค่าเท่ากับ `const mockFn = jest.fn().mockImplementation(()=>value);`

```
const mock = jest.fn<() => number>();
mock.mockReturnValue(42);
mock(); // 42
```

- Mock Module สามารถ mock การทำงานของ module ได้

ใช้ default mock

- *moduleName* ชื่อ module ที่ต้องการ mock

```
jest.mock(moduleName);
```

พร้อม Implementation

```
jest.mock(moduleName, ()=> {
   //… implementation …
})
```

React Testing Library (RTL)



Reaction Testing Library มาพร้อม API สำหรับทำงานกับ React Component

- Render Component

```
render(component)
```

  - *component* Component ที่ต้องการ render ลงไปใน container

```
render(<Home/>)
```

- Queries เป็นกลุ่ม methods ที่มาพร้อม Testing Library สำหรับการค้นหา elements บนหน้า page
  - RTL มาพร้อม queries ชนิดต่าง ๆ ("get", "find", "query") โดยมีความแตกต่างกันตรงที่ method เหล่านั้นจะ throw error ถ้าไม่พบ element หรือไม่ หรือจะ return Promise และ retry
    - เช่น getByPlaceholderText ใช้หา element จาก placeholder text ที่ระบุ หากไม่พบ จะ throw error
  - หลังการ Query เพื่อเลือก element สามารถใช้ Jest และ jest-dom ในการทำ assertion เกี่ยวกับ element
  - หลังการ Query เพื่อเลือก element สามารถใช้ Events API หรือ user-event เพื่อ fire events และ simulate user interactions กับ page

React Testing Library (RTL)



- การ Query Single Element
  - `getBy...` : Return node ที่ match กับ Query และ throw error ถ้าไม่พบ element ที่ match หรือพบมากกว่า 1 match
    - `getByLabelText` , `getByPlaceholderText` , `getByText` , `getByDisplayValue` , `getByRole` , `getByTestId` เป็นต้น
  - `queryBy...` : Return node ที่ match กับ Query และ return null ถ้าไม่พบ element ที่ match และ throw error ถ้าพบ element ที่ match มากกว่า 1 match
    - `queryByLabelText` , `queryByPlaceholderText` , `queryByText` , `queryByDisplayValue` , `queryByRole` , `queryByTestId` เป็นต้น
  - `findBy...` : Returns Promise ที่จะ resolve เมื่อพบ element ที่ match กับ Query โดย promise จะถูก reject ถ้าไม่พบ element หรือ พบมากกว่า 1 match หลังจาก default timeout ที่ 1000ms
    - `findByLabelText` , `findByPlaceholderText` , `findByText` , `findByDisplayValue` , `findByRole` , `queryByTestId` เป็นต้น

React Testing Library (RTL)



- การ Query Multiple Element
  - `getAllBy...` : Return array ของ node ที่ match กับ Query และ throw error ถ้าไม่พบ element ที่ match
    - `getAllByLabelText` , `getAllByPlaceholderText` , `getAllByText` , `getAllByDisplayValue` , `getAllByRole` , `getAllByTestId` เป็นต้น
  - `queryAllBy...` : Return array ของ node ที่ match กับ Query และ return empty array [] ถ้าไม่พบ element ที่ match
    - `queryAllByLabelText` , `queryAllByPlaceholderText` , `queryAllByText` , `queryAllByDisplayValue` , `queryAllByRole` , `queryAllByTestId` เป็นต้น
  - `findAllBy...` : Returns Promise ที่จะ resolve ไปเป็น array ของ element เมื่อพบ element ที่ match กับ Query โดย promise จะถูก reject ถ้าไม่พบ element หลังจาก default timeout ที่ 1000ms
    - `findAllByLabelText` , `findAllByPlaceholderText` , `findAllByText` , `findAllByDisplayValue` , `findAllByRole` , `queryAllByTestId` เป็นต้น

React Testing Library (RTL)



Type of Query	0 Matches	1 Match	>1 Matches	Retry (Async/Await)
Single Element				
`getBy...`	Throw error	Return element	Throw error	No
`queryBy...`	Return `null`	Return element	Throw error	No
`findBy...`	Throw error	Return element	Throw error	Yes
Multiple Elements				
`getAllBy...`	Throw error	Return array	Return array	No
`queryAllBy...`	Return `[]`	Return array	Return array	No
`findAllBy...`	Throw error	Return array	Return array	Yes

React Testing Library (RTL)



- `ByRole` Queries elements จาก role ที่ระบุ
  - เช่น `getByRole('button')`, `getByRole('checkbox')`, `getByRole('img')`, `getByRole('link')` เป็นต้น
  - ตรวจสอบ Role https://www.w3.org/TR/html-aria/#docconformance

- `ByText` Queries elements จาก text ที่ระบุ สำหรับ Text Node
  - เช่น `getByText('About')` เป็นต้น

- `ByPlaceholderText` Queries elements จาก placeholder text ที่ระบุ
  - เช่น `getByPlaceholderText('Username')` เป็นต้น

- `ByDisplayValue` Queries input, textarea, หรือ select element ที่แสดง value ที่ระบุ
  - เช่น `getByDisplayValue('Hello')` เป็นต้น

React Testing Library (RTL)



- ByTestId Queries นิยมใช้ในกรณีที่ต้องการ mark element เอาไว้สำหรับสังเกตค่าเวลาทดสอบ โดยระบุ attribute data-testid ไว้ที่ element

```
<div data-testid="custom-element" />
```

  - เข้าถึง element ด้วย `getByTestId('custom-element')` เป็นต้น

- Screen
  - การ Query ทั้ง `document.body` เป็นรูปแบบการ Query โดยปกติ ดังนั้น DOM Testing Library จึงมาพร้อม `screen` object ที่มีทุก query ที่ pre-bound กับ `document.body`

```
import {render, screen} from '@testing-library/react'

render(
  <div>
    <label htmlFor="example">Example</label>
    <input id="example" />
  </div>,
)

const exampleInput = screen.getByLabelText('Example')
```

React Testing Library (RTL)



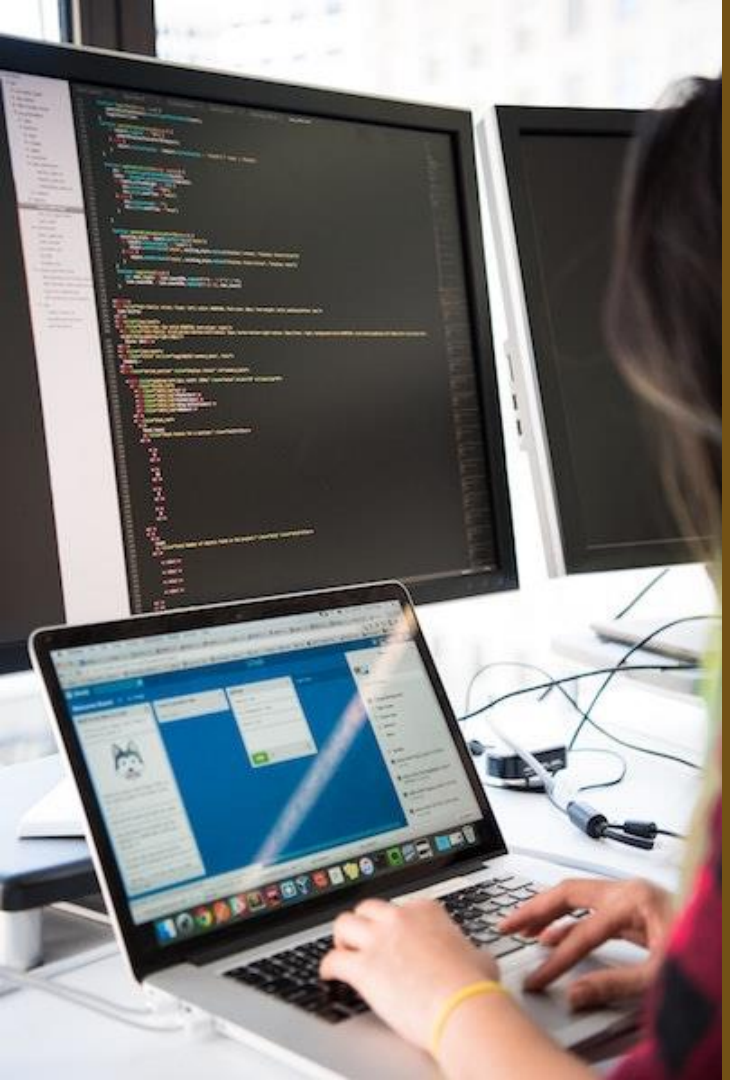
- การ Assert Element ด้วย React Testing Library
  - ด้วย `.toBeInTheDocument()` matcher จาก jest-dom utility library

```
import '@testing-library/jest-dom'
// use `queryBy` to avoid throwing an error with `getBy`
const submitButton = screen.queryByText('submit')
expect(submitButton).not.toBeInTheDocument()
```

  - ด้วย Query และ matcher ต่าง ๆ

```
const submitButton = screen.queryByText('submit')
expect(submitButton).toBeNull() // it doesn't exist
```

```
const submitButtons = screen.queryAllByText('submit')
expect(submitButtons).toHaveLength(0) // expect no elements
```



Workshop 13.1:
Setting Up Testing Library

Setting Up Testing Library

- ใช้ Instruction จาก https://nextjs.org/docs/app/building-your-application/testing/jest
- ใช้ document สำหรับ Application Structure แบบใหม่ที่เป็น App Router



- ติดตั้ง Dev Dependencies Package (`-D` หรือ `--save-dev`) สำหรับใช้ในการทำ Testing

```
npm install -D jest jest-environment-jsdom @testing-library/react @testing-library/dom
@testing-library/jest-dom ts-node @types/jest
```

- เพิ่มการติดตั้ง library `npm install -D @testing-library/user-event ts-jest`
- สังเกต Dev Dependencies และ version ที่สร้างขึ้นมาใน `package.json`
- ใน `package.json` เพิ่ม script สำหรับการ run npm ให้ลองรับการใช้ `jest`

```
"scripts": {
  "dev": "next dev",
  "build": "next build",
  "start": "next start",
  "lint": "next lint",
  "test": "jest",
  "test:watch": "jest --watch"
},
```

เสริม `watch` option สำหรับ rerun test เมื่อมีการเปลี่ยนแปลงขึ้นกับไฟล์

Remarks:



- `jest` version ล่าสุด มี bug ที่ยังไม่ได้รับการแก้ไข
  - มี warning จาก deprecated library `inflight@1.0.6` ซึ่งเป็น dependencies ที่ใช้ใน jest library
  - `inflight@1.0.6` ใช้งาน dependencies library `glob@7.2.3` ซึ่ง deprecated แล้ว

Setting Up Testing Library



- จาก Instruction https://nextjs.org/docs/app/building-your-application/testing/jest
  - สร้างไฟล์ `jest.config.ts` ที่ Project Root Directory ใช้ code จาก instruction



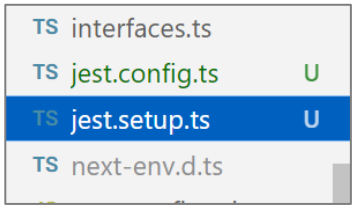
Setting Up Testing Library



- แก้ไขไฟล์ `jest.config.ts` เพื่อระบุ option เพิ่มเติม

```
 9   // Add any custom config to be passed to Jest
10   const config: Config = {
11     coverageProvider: 'v8',
12     testEnvironment: 'jsdom',
13     // Add more setup options before each test is run
14     setupFilesAfterEnv: ['<rootDir>/jest.setup.ts'],
15     preset: 'ts-jest',
16   }
```

- สร้าง `jest.setup.ts` ระดับเดียวกับ `package.json`
  - เพิ่มการใช้ package `@testing-library/jest-dom` ซึ่งรองรับ syntax สำหรับ test case ที่จะใช้

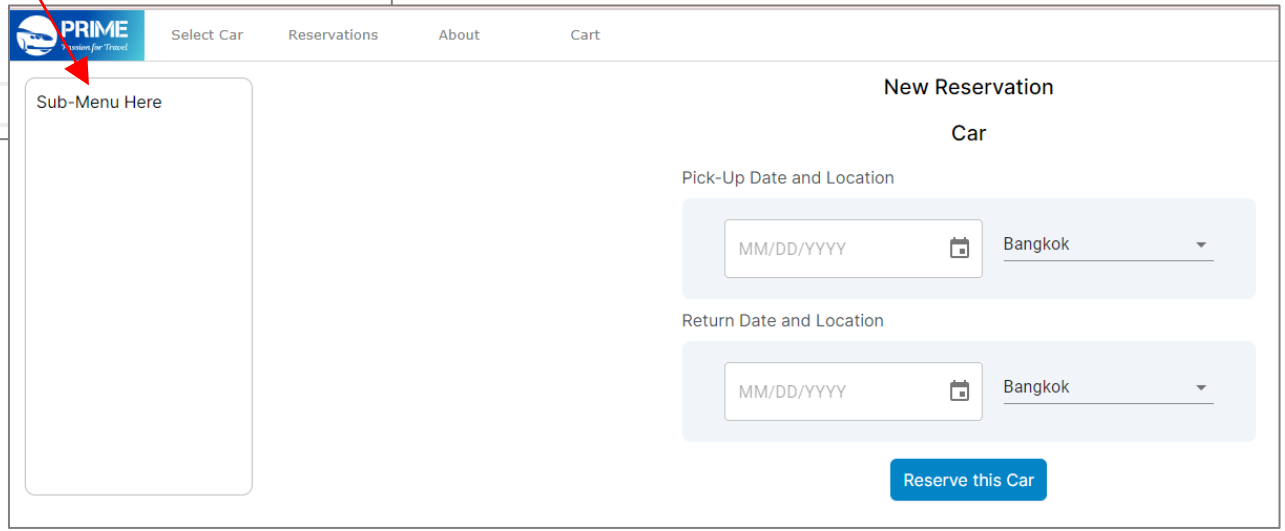




Setting Up Testing Library

- ลองทำ Component Test อย่างง่าย สำหรับ Component `components\ReservationMenu.tsx`

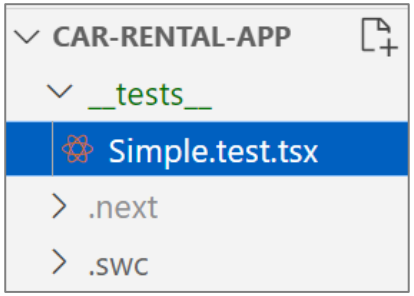
```tsx
export default function ReservationMenu() {
    return (
        <div className={styles.submenu}>
            Sub-Menu Here
        </div>
    );
}
```



Setting Up Testing Library



- สร้างโฟล์เดอร์ `__tests__` ไว้ที่ Root Directory โดย jest จะมาอ่าน test case จากโฟล์เดอร์นี้
  - สร้างไฟล์ `Simple.test.tsx` สำหรับเขียน test script



__tests__ > Simple.test.tsx > ...

```tsx
import ReservationMenu from "@/components/ReservationMenu";
import { render, screen } from "@testing-library/react"

describe('ReservationMenu', ()=>{
    it('should have title', ()=>{
        //Arrange
        render(<ReservationMenu/>)
        //Act
        const bannerText = screen.getByText('Sub-Menu Here')
        //Assert
        expect(bannerText).toBeInTheDocument()
    })
})
```

Test ว่า ReservationMenu มีข้อความว่า "Sub-Menu Here"

Setting Up Testing Library



- ลอง Run Test

```
npm run test
```

```
PS D:\SoftwareDevPrac2\ReactWeb\instructor_wk13\car-rental-app> npm run test

> car-rental-app@0.1.0 test
> jest

 PASS  __tests__/Simple.test.tsx
  ReservationMenu
    √ should have title (22 ms)

Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Snapshots:   0 total
Time:        1.342 s
Ran all test suites.
```

User Event



- @testing-library/user-event รองรับการ fire event เพื่อ interact กับ page หรือ component ใช้งานโดย

```
import userEvent from '@testing-library/user-event'
```

- ตัวอย่าง การสร้างการ Click

```
userEvent.click(screen.getByText('Check'))
expect(screen.getByLabelText('Check')).toBeChecked()
```

```
userEvent.click(elem, {ctrlKey: true, shiftKey: true})
```

- ตัวอย่าง การสร้างการ Double Click

```
const checkbox = screen.getByRole('checkbox')
userEvent.dblClick(checkbox)
expect(onChange).toHaveBeenCalledTimes(2)
expect(checkbox).not.toBeChecked()
```

User Event



- ตัวอย่าง การสร้างการ type

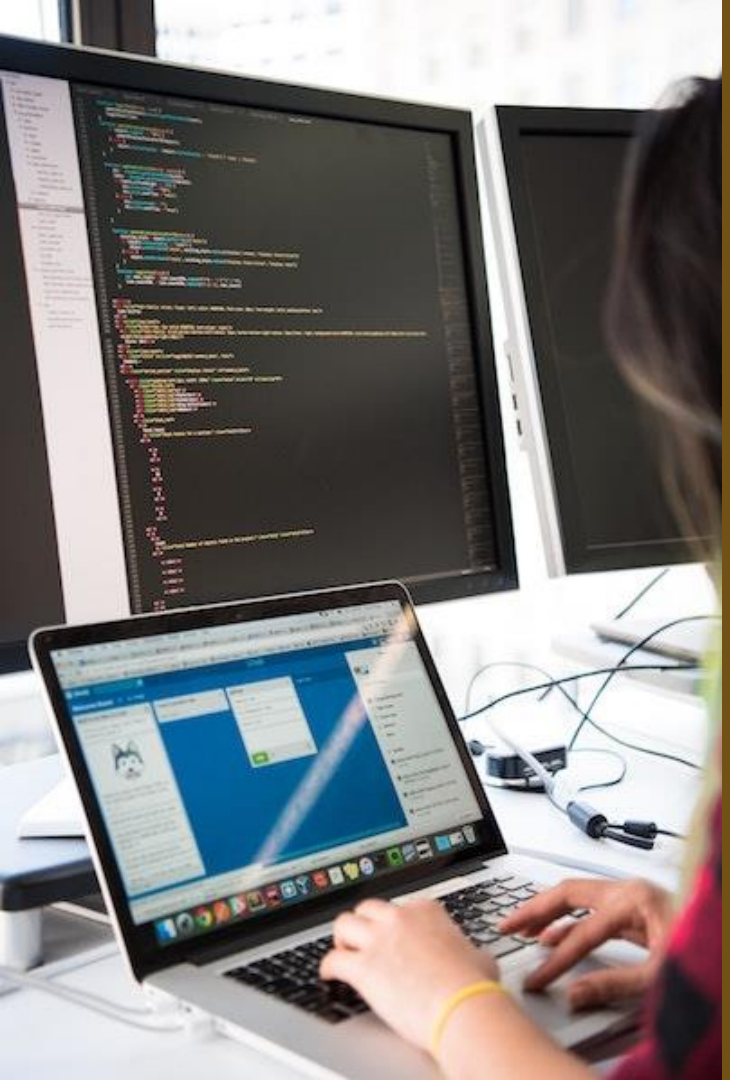
```
userEvent.type(screen.getByRole('textbox'), 'Hello,{enter}World!')
expect(screen.getByRole('textbox')).toHaveValue('Hello,\nWorld!')
```

- ตัวอย่าง การสร้างการ select option

```
test('selectOptions', () => {
  render(
    <select multiple>
      <option value="1">A</option>
      <option value="2">B</option>
      <option value="3">C</option>
    </select>,
  )

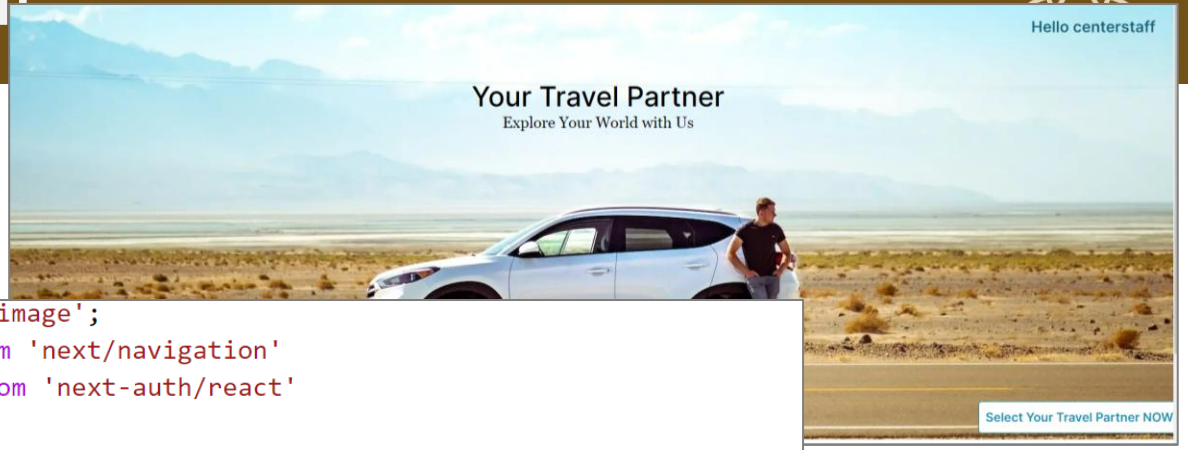
  userEvent.selectOptions(screen.getByRole('listbox'), ['1', '3'])

  expect(screen.getByRole('option', {name: 'A'}).selected).toBe(true)
  expect(screen.getByRole('option', {name: 'B'}).selected).toBe(false)
  expect(screen.getByRole('option', {name: 'C'}).selected).toBe(true)
})
```



Workshop 13.2: Testing User Event

Testing User Event



- ทำ Component Testing สำหรับ Banner.tsx

```tsx
import Image from 'next/image';
import { useRouter } from 'next/navigation'
import { useSession } from 'next-auth/react'

export default function Banner () {
    const covers = ['/img/cover.jpg', '/img/cover2.jpg', '/img/cover3.jpg']
    const [index, setIndex] = useState(0)
    const router = useRouter()
    const { data:session } = useSession()
    console.log(session?.user.token)

    return (
        <div className={styles.banner} onClick={()=>{ setIndex(index+1) }}>
            <Image src={covers[index%3]}
            alt='cover'
            fill={true}
            objectFit='cover'
            />
```

มีการใช้งาน external hook function useRouter และ useSession

รับ click event

Jest Mock

- ในไฟล์ `Simple.test.tsx` เขียน test script สำหรับ Banner

```
import Banner from "@/components/Banner";
import ReservationMenu from "@/components/Res
```

```
        expect(bannerText).toBeInTheDocument()
    })
})

// Mock useRouter
jest.mock('next/navigation', ()=>({
    useRouter() {
        return{
            prefetch: ()=>null
        }
    }
}))

// Mock useSession
jest.mock('next-auth/react', ()=>({
    useSession() {
        return { data: null, user: { name:"Tester" } }
    }
}))
```

Mock `useRouter`

Mock `useSession`

Testing User Event



- ใน `Banner.tsx` เปลี่ยน legacy attribute `objectFit` เป็น Tailwind Style
  - เพื่อหลีกเลี่ยง test warning

```
const { data:session } = useSession()
console.log(session?.user.token)

return (
    <div className={styles.banner} onClick={()=>{ setIndex(index+1) }}>
        <Image src={covers[index%3]}
        alt='cover'
        fill={true}
        className='object-cover'/>
        <div className={styles.bannerText}>
```

- ในไฟล์ `Simple.test.tsx` เขียน test script สำหรับ Banner

```
describe('Banner', ()=>{
    it('should have top banner title', ()=>{
        render(<Banner/>)
        const bannerText = screen.getByText('Your Travel Partner')
        expect(bannerText).toBeInTheDocument()
    })
})
```

Check Banner Title

```
    √ should have title (23 ms)
  Banner
    √ should have top banner title (43 ms)

Test Suites: 1 passed, 1 total
Tests:       2 passed, 2 total
Snapshots:   0 total
Time:        1.337 s
Ran all test suites.
```

Testing User Event



- ในไฟล์ `Simple.test.tsx` เขียน test script สำหรับ Banner

```
describe('Banner', ()=>{
    it('should have top banner title', ()=>{
        render(<Banner/>)
        const bannerText = screen.getByText('Your Travel Partner')
        expect(bannerText).toBeInTheDocument()
    })

    it('should change image when click button', async()=>{
        render(<Banner/>)
        const banner = screen.getByRole('img') as HTMLImageElement;
        await userEvent.click(banner)
        expect(banner.src).toContain('cover2.jpg');
    })
})
```

Test `src` ของ img หลังการ click 1 ครั้ง ต้องเปลี่ยนเป็น path ไปยัง `cover2.jpg`

```
  ReservationMenu
    √ should have title (26 ms)
  Banner
    √ should have top banner title (43 ms)
    √ should change image when click button (103 ms)

Test Suites: 1 passed, 1 total
Tests:       3 passed, 3 total
Snapshots:   0 total
Time:        1.518 s
```

Testing User Event



- ในไฟล์ `Simple.test.tsx` เขียน test script สำหรับตรวจสอบการ click Banner เป็น loop

```
const covers = ['cover.jpg', 'cover2.jpg', 'cover3.jpg']

it('should change image when click button', async()=>{
    render(<Banner/>)
    const banner = screen.getByRole('img') as HTMLImageElement;

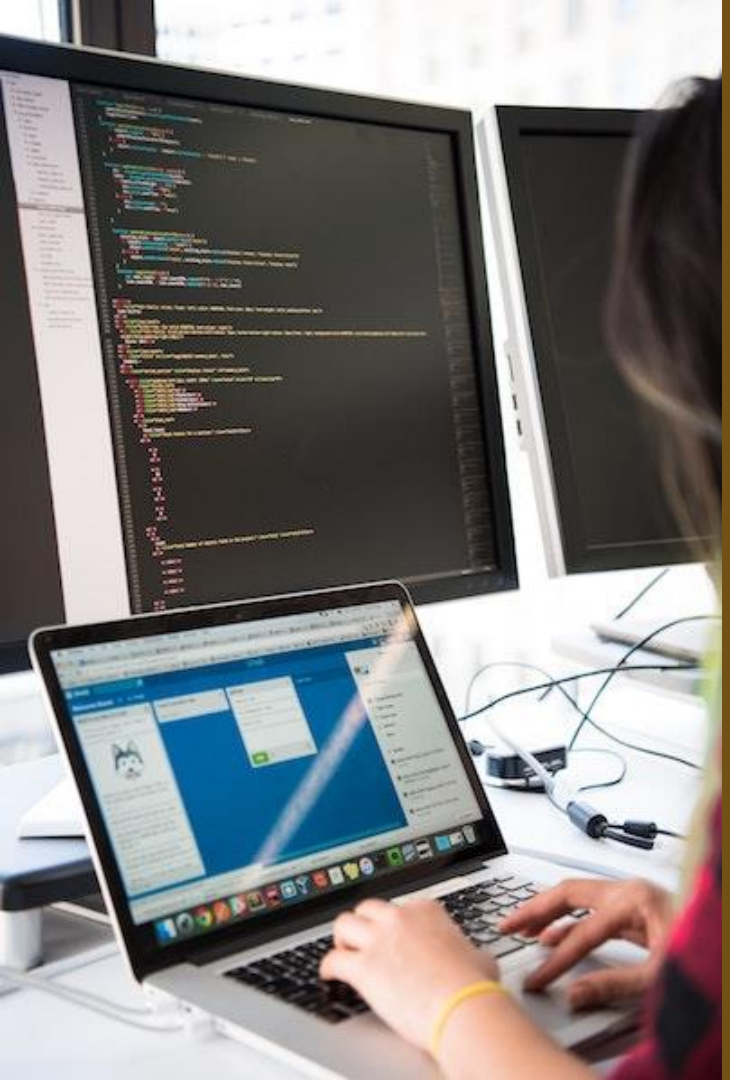
    for (let i=0;i<covers.length;i++) {
        await userEvent.click(banner)
        expect(banner.src).toContain(covers[(i+1)%3]);
    }
})
```

- ลอง Test เฉพาะส่วนที่ describe ว่า Banner

```
npm run test -- -t 'Banner'
```

```
  ReservationMenu
    ○ skipped should have title
  Banner
    √ should have top banner title (135 ms)
    √ should change image when click button (403 ms)

Test Suites: 1 passed, 1 total
Tests:       1 skipped, 2 passed, 3 total
Snapshots:   0 total
Time:        2.881 s, estimated 3 s
Ran all test suites with tests matching "Banner".
```

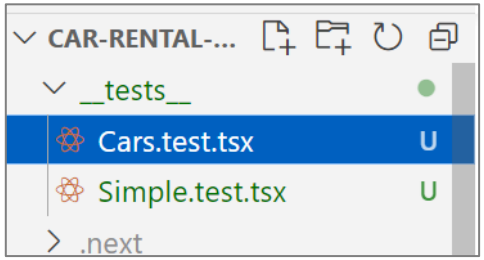


Workshop 13.3:
Mock API
for API Connection Testing

Mock API for API Connection Testing



- ในโฟลเดอร์ `__tests__` เพิ่มไฟล์ `Cars.test.tsx` สำหรับทดสอบการแสดง Car Catalog




```
import CarCatalog from "@/components/CarCatalog"
import { render, screen, waitFor } from "@testing-library/react"

const mockResult = {
    success: true,
    count: 2,
    data: [ {
        _id: "65184657964561dd942a33a6",
        model: "Accord",
        description: "Honda Accord",
        picture: "https://drive.google.com/uc?id=1Vsq3yGo0TbJtNnD-Q-GmIKEPhi774W_O",
        seats: 4,
        doors: 4,
        largebags: 2,
        smallbags: 2,
        automatic: true,
        dayRate: 2500,
        __v: 0,
        id: "65184657964561dd942a33a6"
      },
      {
        _id: "65184756964561dd942a33ab",
        model: "Civic",
        description: "Honda Civic",
        picture: "https://drive.google.com/uc?id=1qZjh3CmMFGEik3lcPFZZCif58q4cqSFe",
        seats: 4,
        doors: 4,
        largebags: 2,
        smallbags: 1,
        automatic: true,
        dayRate: 1800,
        __v: 0,
        id: "65184756964561dd942a33ab"
      }]
}
```

Mock API for API Connection Testing



- เขียน test case เพื่อทดสอบผลการนำ data ไปใส่ car catalog

```
describe('CarCatalog', ()=>{
    it('should have correct number of car images', async ()=>{
        const carCatalog = await CarCatalog({carJson: mockResult})
        render(carCatalog)
        await waitFor(
            ()=> {
                const carImages = screen.queryAllByRole('img')
                expect(carImages.length).toBe(2)
            }
        )
    })
})
```

```
PASS __tests__/Cars.test.tsx
  CarCatalog
    √ should have correct number of car images (195 ms)

Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Snapshots:   0 total
Time:        1.952 s, estimated 2 s
```

- Run test เฉพาะไฟล์

```
npm run test -- Cars.test.tsx
```